

ИИ в проектировании архитектуры и баз данных

Лекция 1.3

Организационные вопросы

1. Приглашение на Github проверяем тут:
 - https://github.com/is-aidd-26/{USERNAME}_project/invitations
2. Все работы сдаются через ASAP
3. Одна неделя — одна итерация
4. Работы проверяются с ИИ-агентом
 - первичная проверка — агент со скиллами и песочницей
 - вычитка и вердикт — преподаватель
 - агент будет усложняться со временем

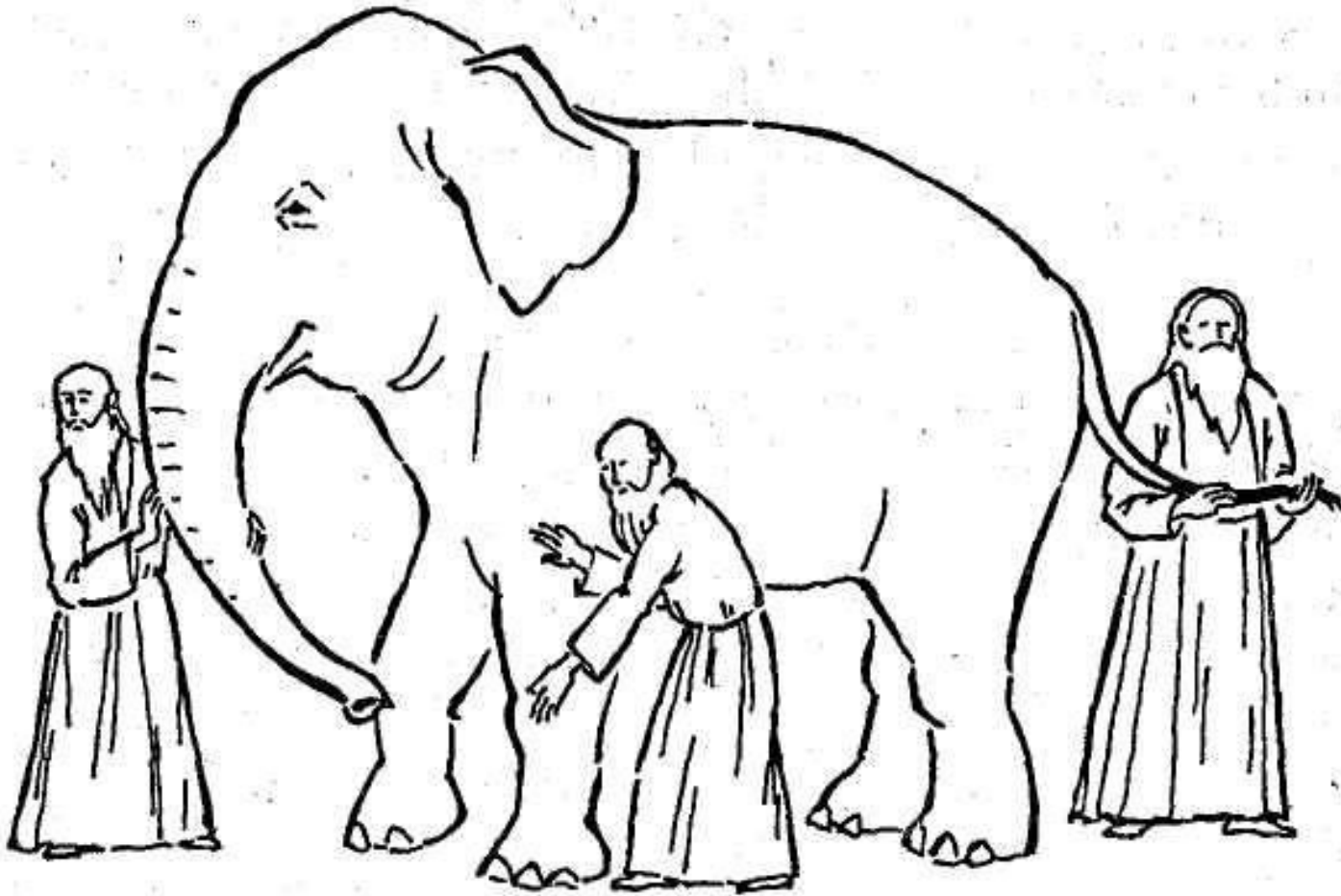
Способы коммуникации

Тип 2. Текстовая коммуникация

1. Сказанное вслух все понимают по-разному
2. Разный темп речи и восприятия

Текст фиксирует смысл

Тип 3. Визуальная коммуникация



Притча о слепых мудрецах

1. Сказанное вслух все понимают по-разному
2. Разный темп речи и восприятия

Текст фиксирует смысл
Визуальный язык даёт
понимание

Цель лекции

Найти способ совместить два языка:

1. Понять систему самому через визуальный язык
2. Составить контракт с ИИ-агентом через текстовый язык

Что обсудим

1. Mermaid
2. Диаграммы архитектуры по модели C4
 - Context
 - Container
 - Component
 - Code
3. ER-диаграмма схемы базы данных
 - концептуальный уровень
 - логический уровень
 - физический уровень
4. Силлы и AGENTS.md
5. Как составлять силлы

Mermaid в markdown

Генерация Mermaid диаграмм

Проблема:

1. Человеку нужен визуальный язык
2. LLM нужен язык разметки

Генерация Mermaid диаграмм

Проблема:

1. Человеку нужен визуальный язык
2. LLM нужен язык разметки

Решение:

1. Mermaid — встраивается в Markdown
2. PlantUML — для сложных схем

Генерация Mermaid диаграмм

Проблема:

1. Человеку нужен визуальный язык
2. LLM нужен язык разметки

Решение:

1. Mermaid — встраивается в Markdown
2. PlantUML — для сложных схем

Промпт на генерацию диаграммы:

1. Язык Mermaid
2. Тип диаграммы
3. Задачу и цель
4. «Старайся соблюдать синтаксис Mermaid»

Генерация ИИ-моделью

правки

диагностика:
ошибки,
предупреждения

Валидация

- mermaid-cli
- линтер mermaid

C4 диаграммы

Диаграммы — модель C4

Визуализация архитектуры системы на 4 уровнях:

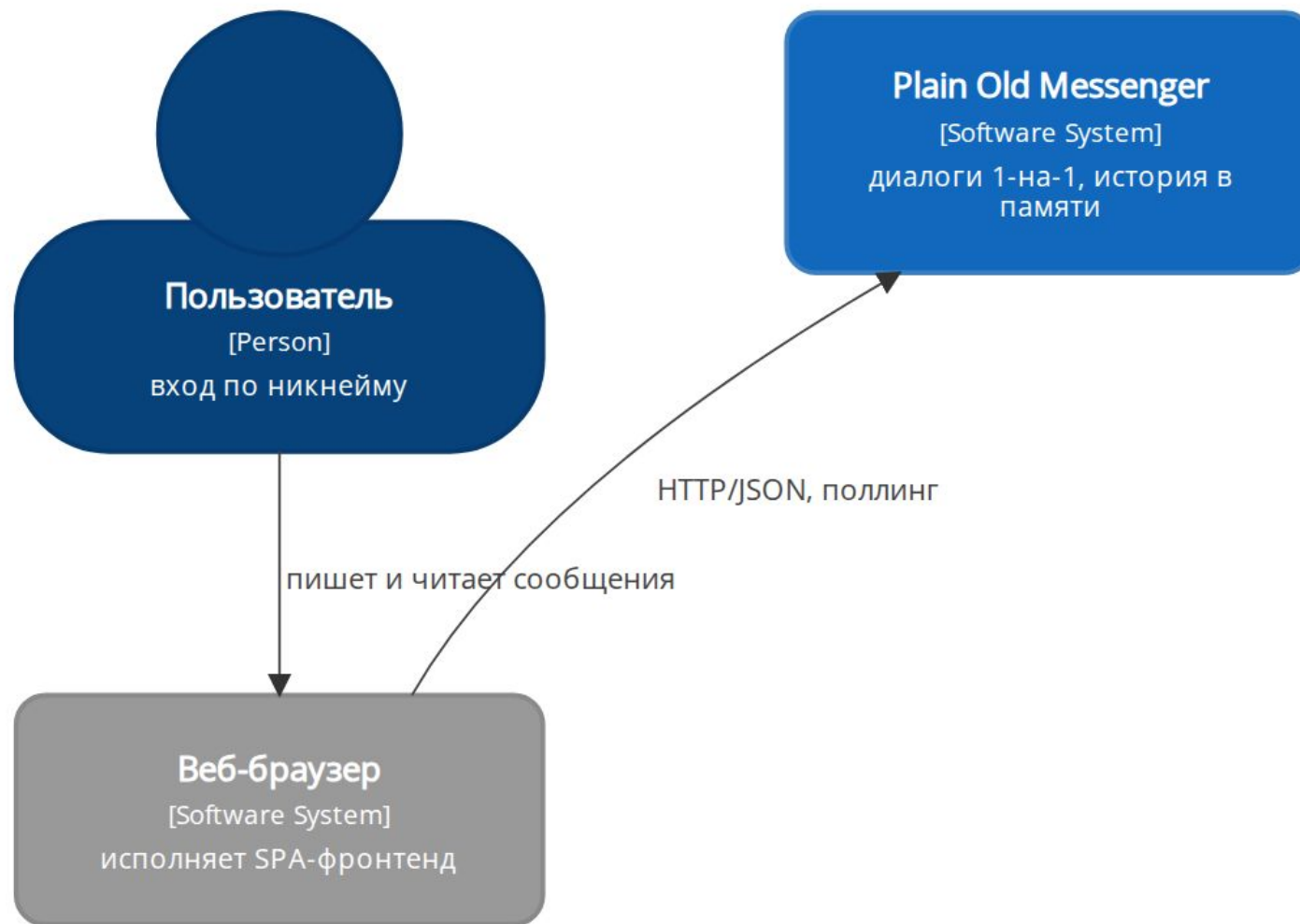
1. **Context** — система и внешний мир
2. **Container** — компоненты системы (единицы развёртывания)
3. **Component** — детализация компонента
4. **Code** — реализация компонента
 - UML Class Diagram

C4 — это не нотация диаграмм, в отличие от UML или ER.

C4 диаграмма контекста

Отображает систему и внешний мир

1. связанные системы
2. взаимодействия

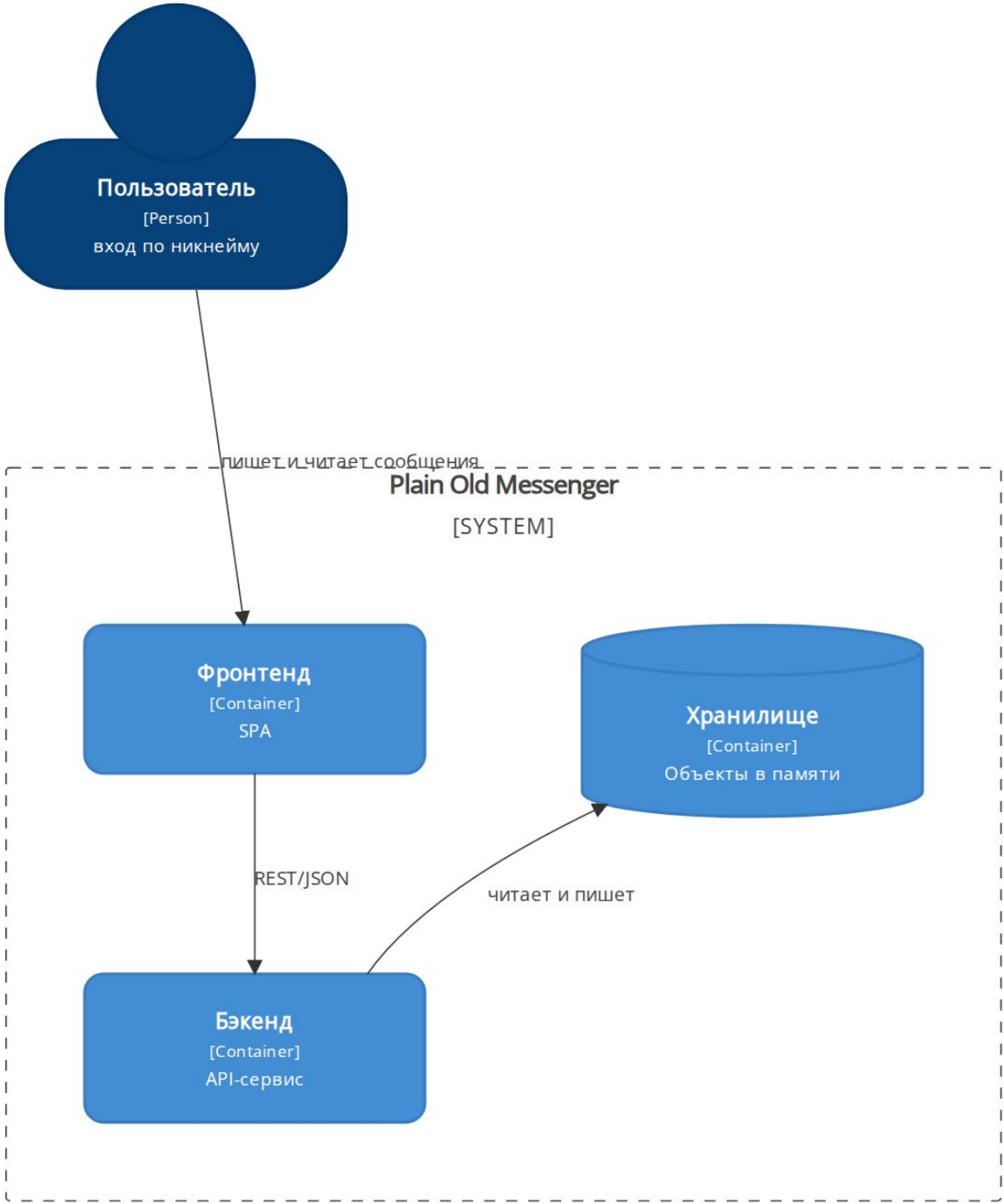


С4 диаграмма контейнеров

Отображает единицы
развёртывания системы

Это всё синонимы:

Компонент	Component
Контейнер	Container
Единица развёртывания	Deployment Unit



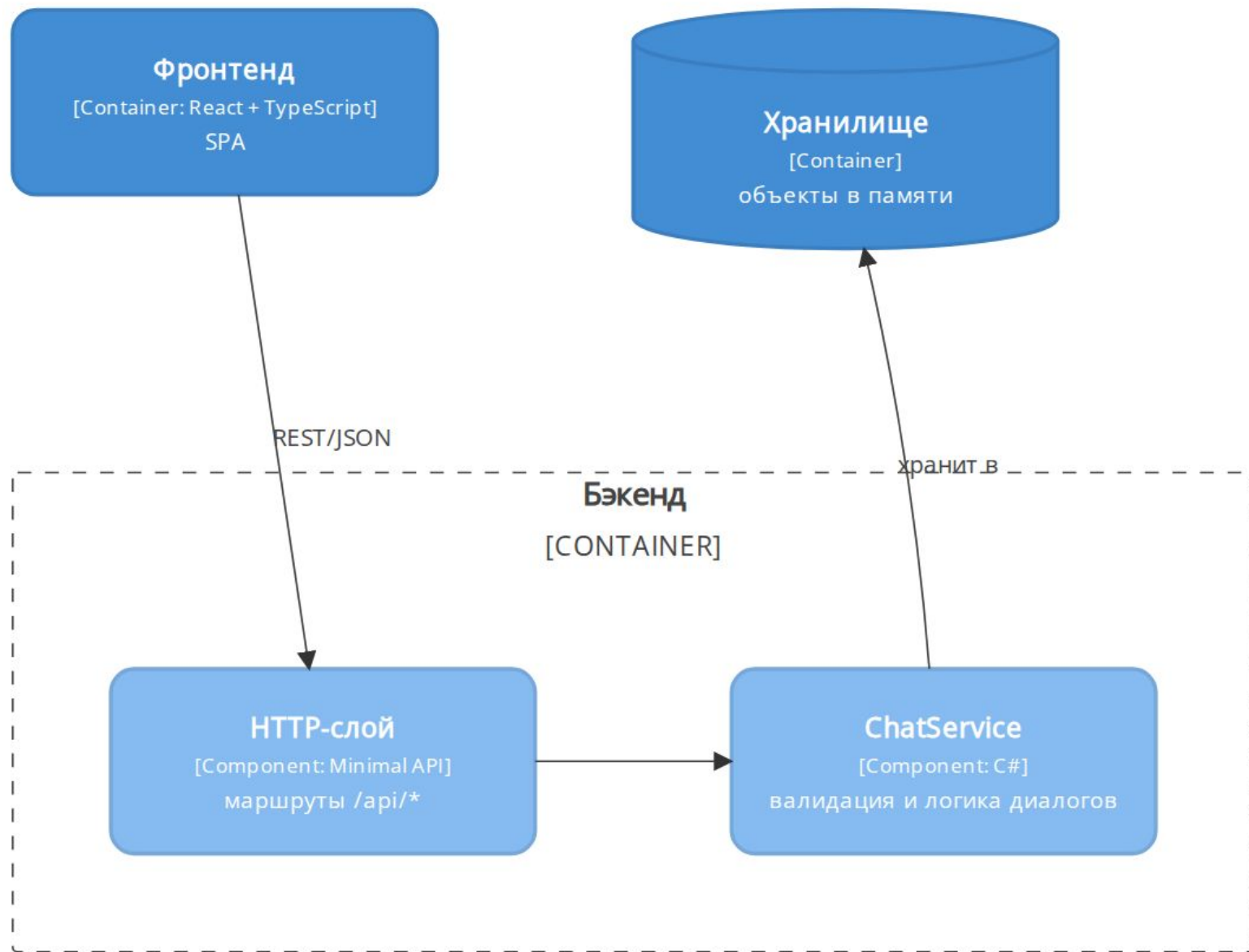
C4 диаграмма компонента

Отображает модули внутри
единицы развёртывания

Модуль = программный компонент

Модулем может быть:

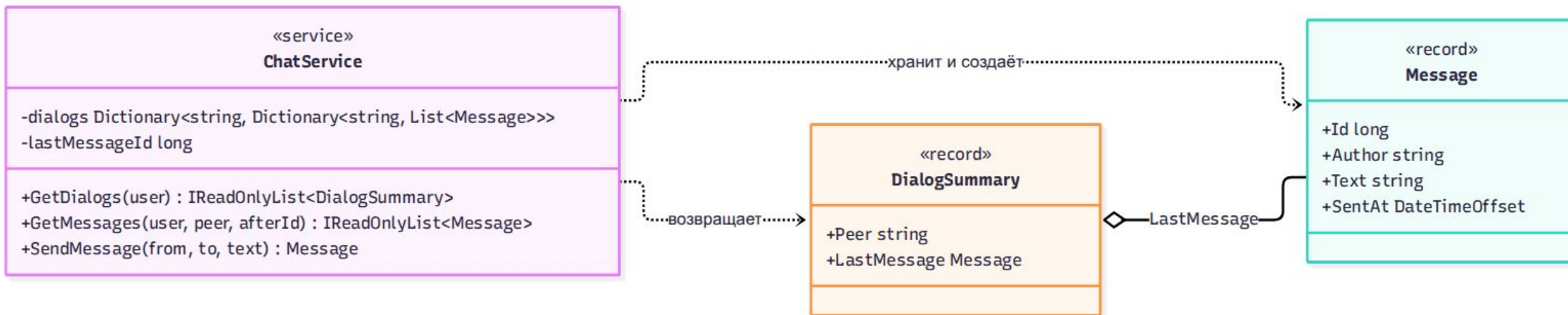
1. пакет / namespace в C#
2. библиотека / assembly в C#



С4 диаграммы кода

Отображает детали реализации
кода

1. Применяем ООП → подойдёт
UML Class Diagram

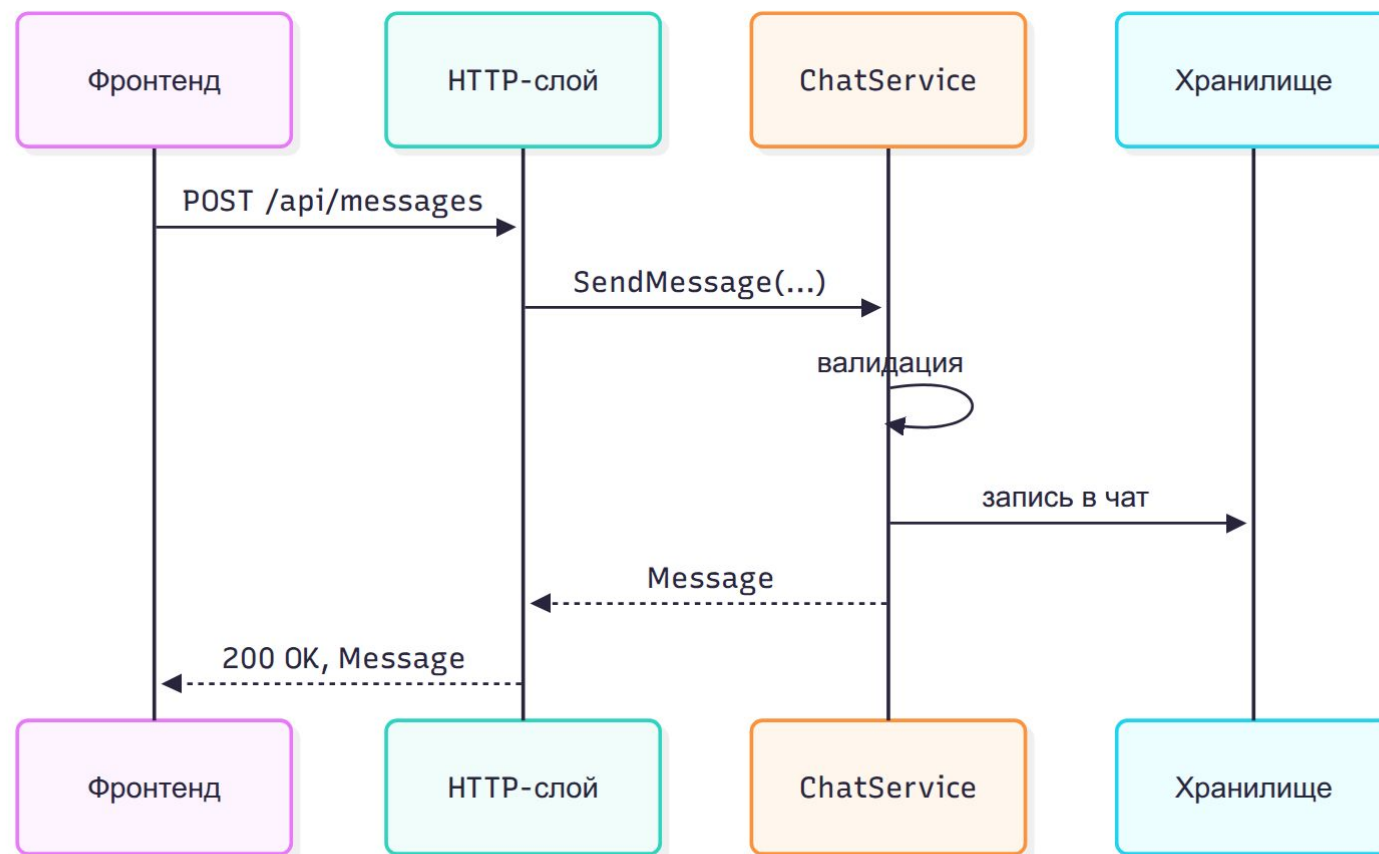


C4 диаграммы кода

Отображает детали реализации кода

1. Применяем ООП → подойдёт UML Class Diagram
2. Есть API → подойдёт UML Sequence Diagram

Отправка сообщения:

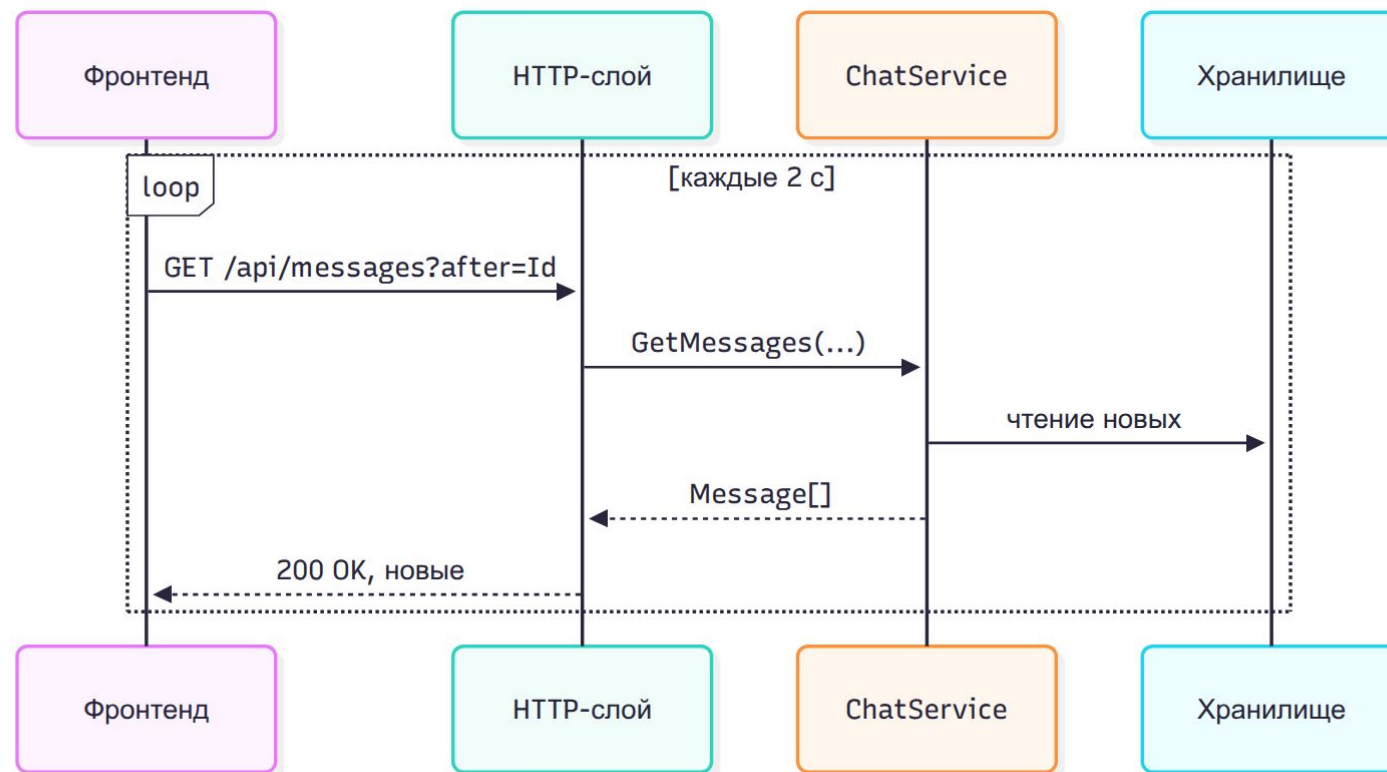


C4 диаграммы кода

Отображает детали реализации кода

1. Применяем ООП → подойдёт UML Class Diagram
2. Есть API → подойдёт UML Sequence Diagram

Получение сообщения:



ER диаграммы

ER-диаграмма

ER — это Entity-Relation

ER-диаграмма

ER — это Entity-Relation

Три уровня проектирования схемы БД:

1. **Концептуальный** — определить сущности и связи
 - определить кардинальность связей — 1:1, 1:N, M:N

ER-диаграмма

ER — это Entity-Relation

Три уровня проектирования схемы БД:

1. **Концептуальный** — определить сущности и связи
 - определить кардинальность связей — 1:1, 1:N, M:N
2. **Логический** — привести схему к реляционной модели
 - нет отношения M:N — нужны соединительные таблицы
 - нужны ключи — один primary, сколько угодно unique
 - есть внешние ключи — foreign keys

ER-диаграмма

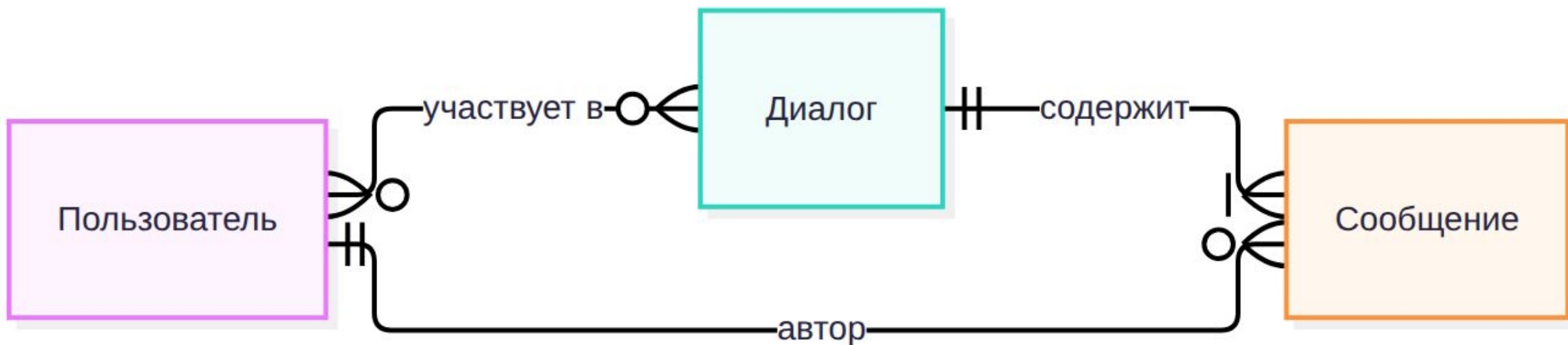
ER — это Entity-Relation

Три уровня проектирования схемы БД:

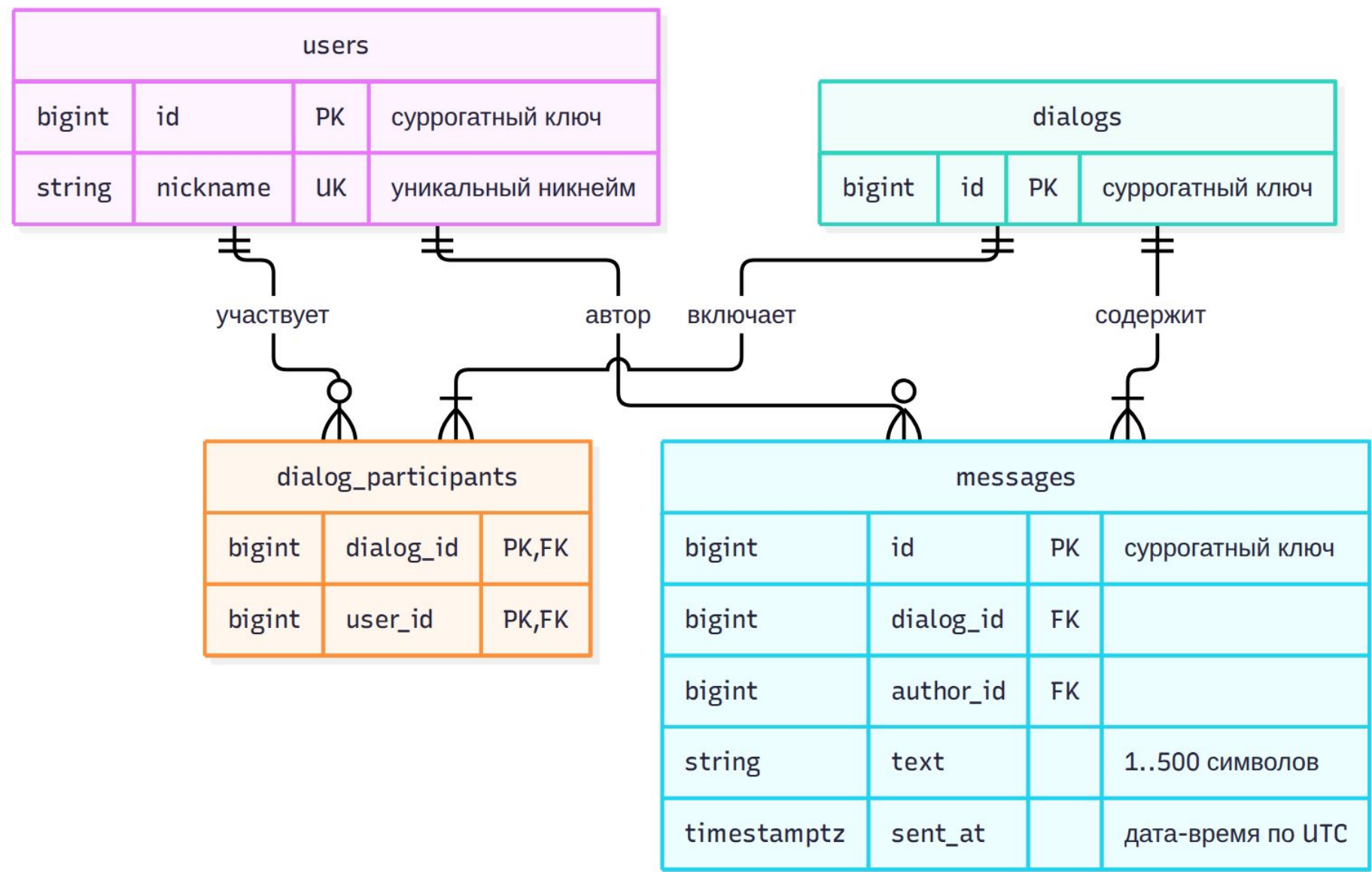
1. **Концептуальный** — определить сущности и связи
 - определить кардинальность связей — 1:1, 1:N, M:N
2. **Логический** — привести схему к реляционной модели
 - нет отношения M:N — нужны соединительные таблицы
 - нужны ключи — один primary, сколько угодно unique
 - есть внешние ключи — foreign keys
3. **Физический** — учесть особенности СУБД
 - конкретные типы данных
 - конкретные ограничения целостности — primary key, unique key, foreign key, not null, check

ER-диаграмма — концептуальный уровень

Концептуальная модель данных Plain Old Messenger



ER-диаграмма — логический уровень



ER-диаграмма — физический уровень

SQL идеально подходит

```
CREATE TABLE dialog_participants (  
    dialog_id    bigint    NOT NULL,  
    user_id      bigint    NOT NULL,  
  
    PRIMARY KEY (dialog_id, user_id),  
    FOREIGN KEY (dialog_id) REFERENCES dialogs (id) ON DELETE RESTRICT,  
    FOREIGN KEY (user_id) REFERENCES users (id) ON DELETE RESTRICT  
);
```

AGENTS.md и SKILL.md

Что такое skill

Ещё один способ «продать» markdown

Что такое skill

~~Ещё один способ «продать» markdown~~

Markdown-файлы, загружаемые в контекст сессии

	OpenCode	Claude Code
Загружается всегда	AGENTS.md	AGENTS.md, CLAUDE.md

Что такое skill

~~Ещё один способ «продать» markdown~~

Markdown-файлы, загружаемые в контекст сессии

	OpenCode	Claude Code
Загружается всегда	AGENTS.md	AGENTS.md, CLAUDE.md



Раньше был трюк с CLAUDE.md:
@AGENTS.md

Сейчас Claude Code «понимает»
AGENTS.md

Что такое skill

~~Ещё один способ «продать» markdown~~

Markdown-файлы, загружаемые в контекст сессии

	OpenCode	Claude Code
Загружается всегда	AGENTS.md	AGENTS.md, CLAUDE.md
Загружается по требованию • /skill-name в чате • инструмент skill у агента	<code>.agents/skills/<name>/SKILL.md</code> <code>.opencode/skills/<name>/SKILL.md</code> <code>.claude/skills/<name>/SKILL.md</code>	<code>.claude/skills/<name>/SKILL.md</code>

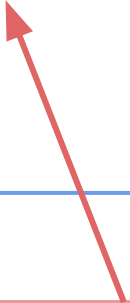
Что такое skill

~~Ещё один способ «продать» markdown~~

Markdown-файлы, загружаемые в контекст сессии

	OpenCode	Claude Code
Загружается всегда	AGENTS.md	AGENTS.md, CLAUDE.md
Загружается по требованию • /skill-name в чате • инструмент skill у агента	<code>.agents/skills/<name>/SKILL.md</code> <code>.opencode/skills/<name>/SKILL.md</code> <code>.claude/skills/<name>/SKILL.md</code>	<code>.claude/skills/<name>/SKILL.md</code>

Можно сделать symlink



Что такое skill

~~Ещё один способ «продать» markdown~~

Markdown-файлы, загружаемые в контекст сессии

	OpenCode	Claude Code
Загружается всегда	AGENTS.md	AGENTS.md, CLAUDE.md
Загружается по требованию • /skill-name в чате • инструмент skill у агента	<code>.agents/skills/<name>/SKILL.md</code> <code>.opencode/skills/<name>/SKILL.md</code> <code>.claude/skills/<name>/SKILL.md</code>	<code>.claude/skills/<name>/SKILL.md</code>
Загружается по желанию агента	любой файл в проекте	любой файл в проекте

Составляем skill

Общие принципы

1. AGENTS.md и скиллы пишет только человек
 - можно сделать черновик ИИ-генерацией и переписать
 - можно переписать с другого проекта
 - можно написать с нуля

Общие принципы

1. AGENTS.md и скиллы пишет только человек
 - можно сделать черновик ИИ-генерацией и переписать
 - можно переписать с другого проекта
 - можно написать с нуля
2. Только релевантный контент
 - AGENTS.md от 10 до ~200-300 строк, меньше — лучше
 - SKILL.md — аналогично

Общие принципы

1. AGENTS.md и скиллы пишет только человек
 - можно сделать черновик ИИ-генерацией и переписать
 - можно переписать с другого проекта
 - можно написать с нуля
2. Только релевантный контент
 - AGENTS.md от 10 до ~200-300 строк, меньше — лучше
 - SKILL.md — аналогично
3. При ошибках агента дополнять правила
 - можно попросить агента сделать выводы и отредактировать AGENTS.md / SKILL.md
 - ... а потом переписать самому

Структура скилла

Минималистичный вариант:

1. Workflow

- задавай вопросы, чтобы устранить неоднозначности
- валидируй через mermaid-cli

2. Style

- минимум деталей в надписях
- помещается на слайд

3. Guidelines

- что делать, если...
- чего не делать

Способы написать скилл

1. Сгенерировать через ИИ
2. Начать с малого, дописывать при ошибках модели
3. Собрать в скилл существующий подход

Способы написать скилл

- ~~1. Сгенерировать через ИИ~~
2. Начать с малого, дописывать при ошибках модели
3. Собрать в скилл существующий подход

Способы написать скилл

- ~~1. Сгенерировать через ИИ~~
2. Начать с малого, дописывать при ошибках модели
3. Собрать в скилл существующий подход
 - Выбрать методологию: Canon TDD, «Принципы Unit-тестирования» и т.д.
 - Сделать выжимку самому и/или с помощью ИИ
 - Пример: <https://github.com/EvilFreelancer/logika>

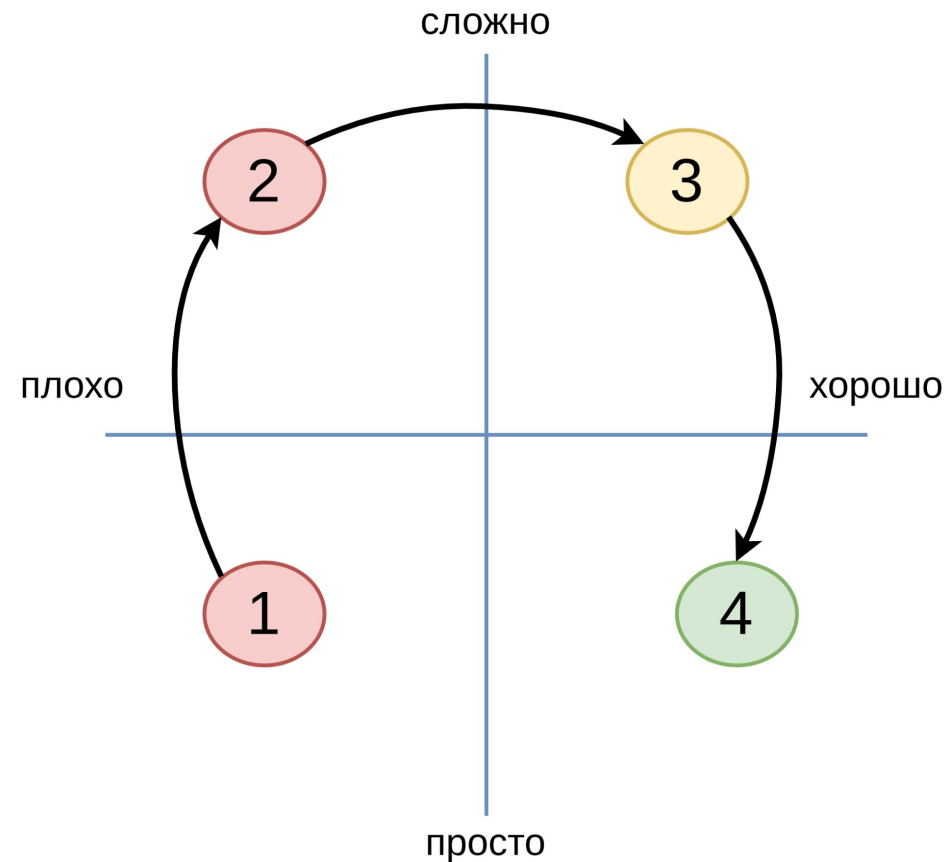
Ссылки

1. Пример этого курса: github.com/is-aidd-26/plain-old-messenger
 - Все диаграммы к лекции сделаны тут
 - Сохранены скиллы для диаграмм
2. Пример скилла-выжимки: github.com/EvilFreelancer/logika

Цикл Репина

«Сначала художник рисует просто и плохо. Потом сложно и плохо. Потом сложно и хорошо. И только потом просто и хорошо.»

Илья Репин



Конец!

Вопросы?